# Course Handbook

Applied AI and Automation for Executives | KLAi Professional Learning, Kampala | Instructor: Jotham Martin Wambi

This handbook is yours to keep. It gives you the words and the pictures behind the course, so that you arrive at Session 1 able to follow every idea, and so that you can come back to it whenever a term needs a reminder. Read it in any order. Parts 3 to 15 explain how AI works and how to judge it, using the standard terms of the field, each explained when it first appears. Part 16 tells you what to bring. Part 17 is a glossary with the session where each term returns. Part 18 is the fourteen sessions on one page.

## What is inside


| Part | Title |
|---|---|
| **1** | Welcome and how the course works |
| **2** | Why this introduction matters |
| **3** | AI, machine learning, deep learning and generative AI |
| **4** | How machines learn |
| **5** | How a model is trained |
| **6** | Data: the fuel and the risk |
| **7** | Language models and generative AI |
| **8** | Where AI is used in industry, and what suits it |
| **9** | How we know a model is good |
| **10** | Benchmarks |
| **11** | Training and inference |
| **12** | What happens when you press send |
| **13** | Foundation models, open models, fine-tuning and cost |
| **14** | Is our data ready for AI? |
| **15** | From model to working product: workflows, agents and tools |
| **16** | What to bring to Session 1 |
| **17** | Glossary |
| **18** | The fourteen sessions at a glance |


**A note on simplification**

This handbook explains ideas in plain words. Where it simplifies, it says so with the word "simplified". The simple version is correct enough to make good decisions. Specialists add detail that you do not need for the work of this course.
All company names and figures in examples are fictional or general. The handbook contains no statistics.

Part 1

# Welcome and how the course works

The course has fourteen sessions of two hours each, 28 hours in all. You choose the dates and the place with the instructor: in person, online or at your workplace. The group is small, from one to five executives, so the time is spent on the work you do.

## Two kinds of session

Sessions 1 to 11 explain how AI works, one idea at a time: how a machine sees, how it learns from examples, how a model is measured, how words become numbers, how language models, foundation models and open models work, how to prompt and check them, and how agents act. In each of these sessions the instructor runs live demonstrations while you watch, and you may join in on your own laptop. Each session also has a short paper exercise that needs no computer. There is no coding.

Sessions 12 to 14 are practicals. You work on your own organisation, with the instructor beside you: you map one workflow and work out the hours it could save, you write a first AI use policy and a vendor question list, and you assess how ready your information is and plan the next 90 days.

## Nothing to prepare

There is nothing to prepare before Session 1 except reading this handbook. Bring a laptop and an AI assistant account you already use, so that you can join the demonstrations if you wish. Use invented or sample information only.

## No tests and no marks

There are no tests and no required homework. Anything you do between sessions is optional. You show progress by what you understand and what you keep.

## What you keep

- A one-page take-away sheet from every session, such as a distance cheat sheet, a decision tree for giving a model your knowledge and a permissions table for agents.
- From the practicals: your workflow map and a costed business case, a first AI use policy, a risk register and a vendor question list.
- A readiness scorecard, a data roadmap and a 90-day plan.
- The confidence to question a vendor and to supervise a pilot.

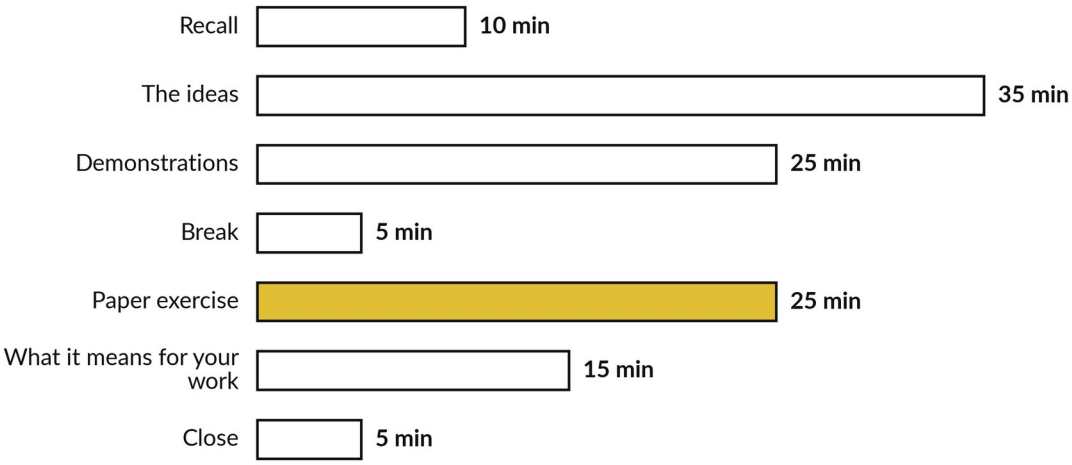
## Automation tools

The course shows two example automations at work on the same twelve customer messages: one that routes them by plain rules, and one where an AI assistant sorts them and a person checks. You do not build either. Other tools exist and Part 15 describes some of them as options, so that you can choose well when the time comes.

## After the course

The instructor follows up with you at 30, 60 and 90 days. You return a one-page check-in form and a results tracker. The follow-up asks what is running, how much time it saves and what changed in your policy.

## How a session runs




*The usual shape of a two-hour session in Sessions 1 to 11. In the practicals, Sessions 12 to 14, about 55 minutes go to your own work.*

Part 2

# Why this introduction matters

You cannot judge a claim, a vendor or a result unless you have the words and a picture of how the thing works. A vendor who says "our model learns from your data" or "the agent acts on its own" is saying something specific. With the picture in your head you can ask what data, learns how, and acts on what. Without the picture you can only nod.

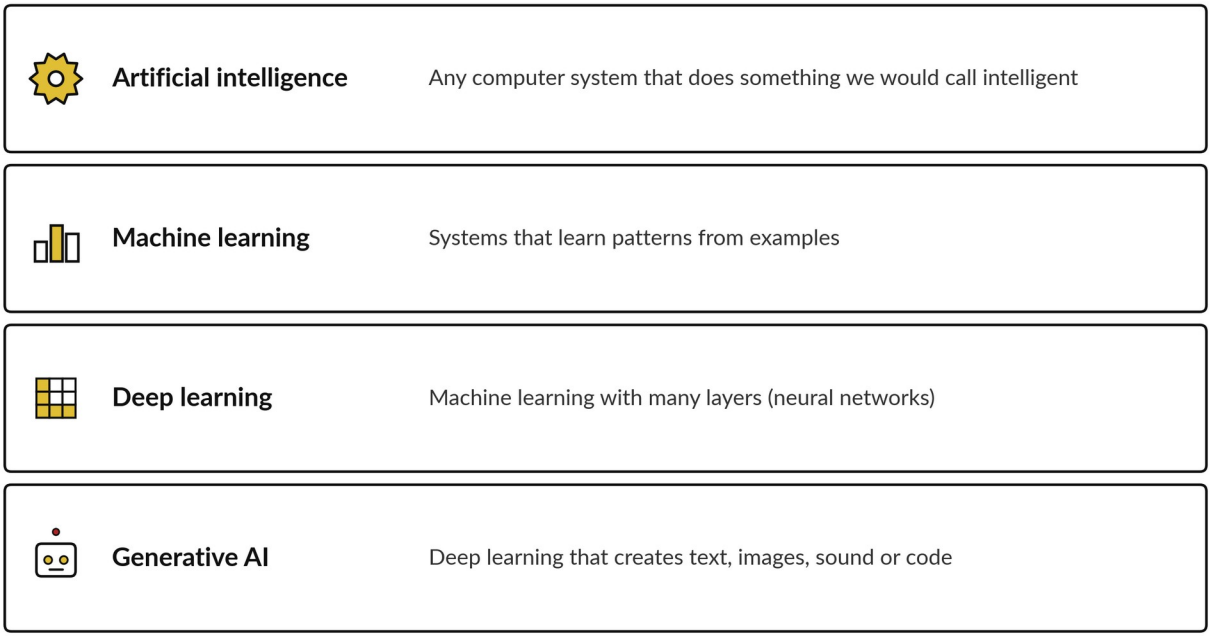
This handbook gives you that picture once. Every idea in it comes back later in the course, taught in more depth.

| Idea in this handbook | Where it returns |
|---|---|
| **Weighted sum, weight, bias, parameters, loss, gradient descent** | Sessions 0 and 1: how a model is a weighted sum with adjustable numbers, and how training adjusts them |
| **Training, validation and test sets** | Sessions 0 and 3: honest scores and overfitting |
| **k-nearest neighbours and cross-validation** | Session 2: learning from examples, and testing without fooling yourself |
| **Data quality dimensions and the data inventory** | Session 14: your data inventory and readiness scorecard |
| **Bias, privacy and personal data** | Session 13: risk register and policy; Session 14: sensitive data |
| **Token, softmax, temperature, self-attention, transformer** | Session 5: how a language model predicts the next token; Session 8: prompting |
| **Hallucination, retrieval-augmented generation (RAG)** | Session 9: giving a model your documents; Session 10: checking answers |
| **Prompt injection** | Sessions 10 and 11: limits on what an agent may do |
| **Workflow, trigger and human-in-the-loop approval** | Session 12: mapping a workflow |
| **Agents, permissions, least privilege and guardrails** | Session 11: the Tool Boundary Sheet, a permissions table for an agent |
| **Baseline, pilot and ROI** | Session 12: the hours-saved estimate and the business case |
| **Foundation model, open weights, quantisation** | Sessions 6 and 7: model families and open weights; Session 13: questions for vendors |

Part 3

# AI, machine learning, deep learning and generative AI

These four terms are often used as if they mean the same thing. They do not. Each is a smaller part of the one before, like boxes that sit inside each other.




*The nested picture. Each layer is part of the one above it. Simplified: the boundaries are fuzzy in practice.*

| Term | A business example |
|---|---|
| **Artificial intelligence (AI)** | A set of written rules that sorts an email into spam if it contains the word "lottery". This is old-style AI: a person wrote every rule. |
| **Machine learning (ML)** | A spam filter that has studied thousands of emails already marked spam or not spam, and learned which words tend to go with spam. Nobody wrote the rules by hand. |
| **Deep learning** | A system that reads a photo of a handwritten digit and says which digit it is. It uses a neural network with many layers, which suits photos, sound and language. |
| **Generative AI** | An assistant that drafts a reply to a customer, summarises a meeting or writes a first version of a policy. It generates something new each time. |

Most of what you will use in this course is generative AI, built on deep learning, which is a kind of machine learning, which is a part of AI. When a vendor says "AI", ask which of the four they mean.

Part 4

# How machines learn

A machine learns from experience, which for a machine means data. There are four main types of learning. Each suits a different problem. The first example in each row is from everyday life.

| Type of learning | How it learns | Examples |
|---|---|---|
| **Supervised learning** | The machine studies examples that come with the correct answer, called the label. It learns to give the right answer for new cases. | Everyday: spam or not spam; a house price from its floor area and bedrooms.<br>At work: sort a customer message as order, payment or complaint. |
| **Unsupervised learning** | No labels are given. The machine finds groups and structure in the data on its own. | Everyday: grouping songs that sound alike.<br>At work: grouping 300 shops into types of buyer by how and when they order. |
| **Reinforcement learning** | The machine tries actions and receives a reward or a penalty. Over many tries it learns which actions pay off. | Everyday: a program learning to play a board game.<br>At work: choosing delivery routes that save fuel and arrive on time. |
| **Self-supervised learning** | The machine predicts a hidden part of its own data, so the data supplies the labels. | Everyday: phone autocomplete predicting the next word of a sentence.<br>At work: the first stage of every language model. |

## Words you will meet often

- **Feature, label, example.** A feature is one input measurement, such as the floor area of a house. The label is the correct answer, such as its price. An example (or training example) is one row of data: the features and the label together.
- **Classification** predicts a category as the answer: spam or not spam, urgent or routine.
- **Regression** predicts a number as the answer: a house price, next week's demand, a delivery time.
- **Clustering** is the unsupervised cousin: it makes the groups itself, without being told the categories.
- **Decision tree.** A model made of yes-or-no questions on features. Each question is a split, each final answer is a leaf, and depth is the number of questions on the longest path. Example: should you carry an umbrella? Is the sky cloudy? If yes, is the chance of rain over 50 percent? Session 2 builds one from twelve past days.
- **k-nearest neighbours (k-NN).** To predict for a new case, find the k most similar training examples, measured with a distance metric, and take their answer. Example: a new house of 80 square metres, 3 km from town. Its 3 nearest sold neighbours cost UGX 250, 270 and 190 million, so the estimate is the average, (250 + 270 + 190) / 3 = 237 million. For a category, the neighbours take a majority vote.

Simplified: real systems often mix these types. A language model, for example, learns by self-supervised learning first and is then improved with other methods (Part 7).

Part 5

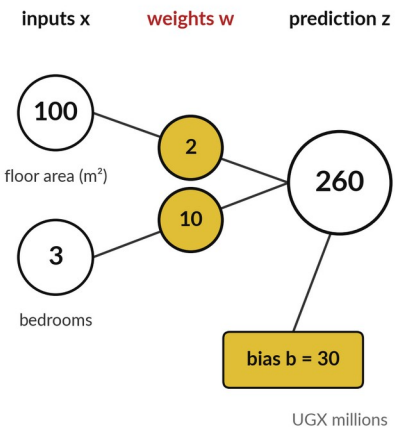
# How a model is trained

A model is a program whose behaviour comes from numbers it has learned. No person typed in the rules. Training is the process of finding good numbers. We start with a case anyone knows: predicting the price of a house from its floor area and its number of bedrooms.

## A weighted sum: the smallest model

Each input, called a feature, is multiplied by a number called a weight, which says how much that input counts. The results are added, and one more number, the bias, is added once. This is a weighted sum: output = w1 x1 + w2 x2 + b. Here x1 is the floor area in square metres, x2 is the number of bedrooms, w1 and w2 are their weights, and b is the bias, a number that shifts the result up or down whatever the inputs are.

Let w1 = 2 (UGX 2 million per square metre), w2 = 10 (UGX 10 million per bedroom) and b = 30 (a base price of UGX 30 million). For a house of 100 square metres with 3 bedrooms the weighted sum is 2 x 100 + 10 x 3 + 30 = 260, so the predicted price is UGX 260 million. If the house really sold for UGX 300 million, the error is 300 - 260 = 40 million.




**The weighted sum**

$z = w_1x_1 + w_2x_2 + b$

$z = \mathbf{w} \cdot \mathbf{x} + b$ (vector form)

$z = 2 \times 100 + 10 \times 3 + 30 = 260$

Then an activation f gives a = f(z):

ReLU: $f(z) = \max(0, z)$

Sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$

x input, w weight, b bias, z sum

actual 300, error = 300 − 260 = 40

*A house price as a weighted sum. The weights and the bias together are the parameters of the model.*

## Parameters

The weights and the biases of a model are together called its parameters: the numbers that training adjusts. The model above has three. The large language models behind today's assistants have billions.

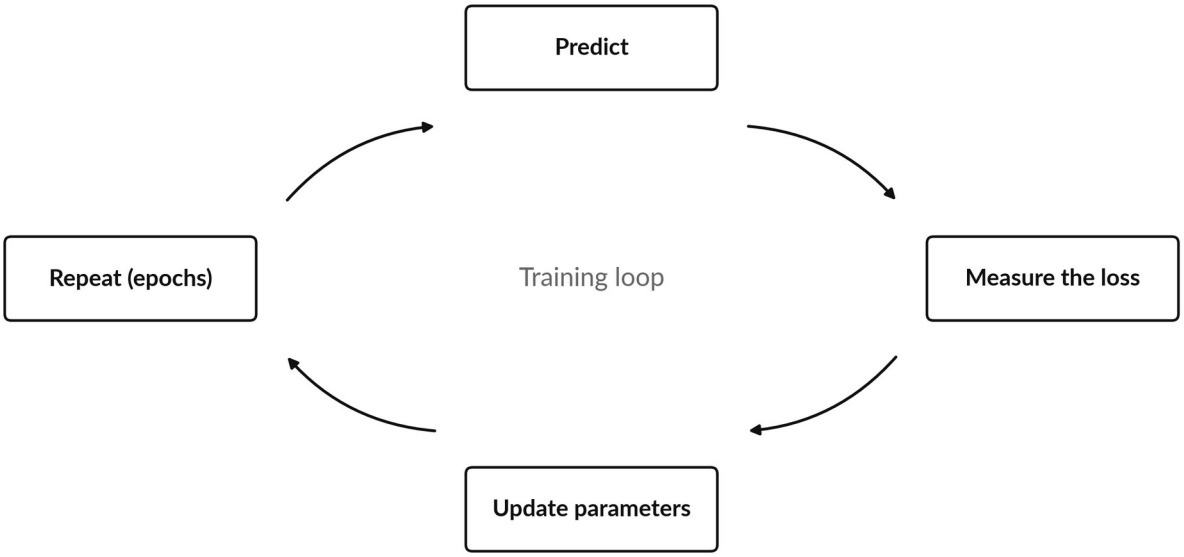
## Neuron and activation function

A neuron is the same weighted sum with one more step. It computes the weighted sum, adds the bias, and applies an activation function: a simple function that turns the result into the neuron's output and adds non-linearity. Everyday example: should you carry an umbrella? The inputs are the chance of rain in percent and the minutes you will be outside. Take the weights 0.1 and 0.05 and the bias −6. With a 70 percent chance and 30 minutes outside: 0.1 x 70 + 0.05 x 30 - 6 = 7.0 + 1.5 - 6 = 2.5. The activation function here is a step: above 0 the output is 1 (carry it), otherwise 0. So you carry it. With a 20 percent chance: 2.0 + 1.5 - 6 = −2.5, below 0, so the output is 0 and the umbrella stays at home.

Real networks use smoother activation functions. ReLU (rectified linear unit) outputs 0 for negative values and the value itself otherwise. The sigmoid function squeezes any value into the range 0 to 1. Without an activation function, stacked layers would collapse into a single weighted sum, so it is the activation function that lets a network learn curved patterns. A layer is a set of neurons that take the same inputs. Layers between the input and the output are hidden layers.

## The training loop and the loss

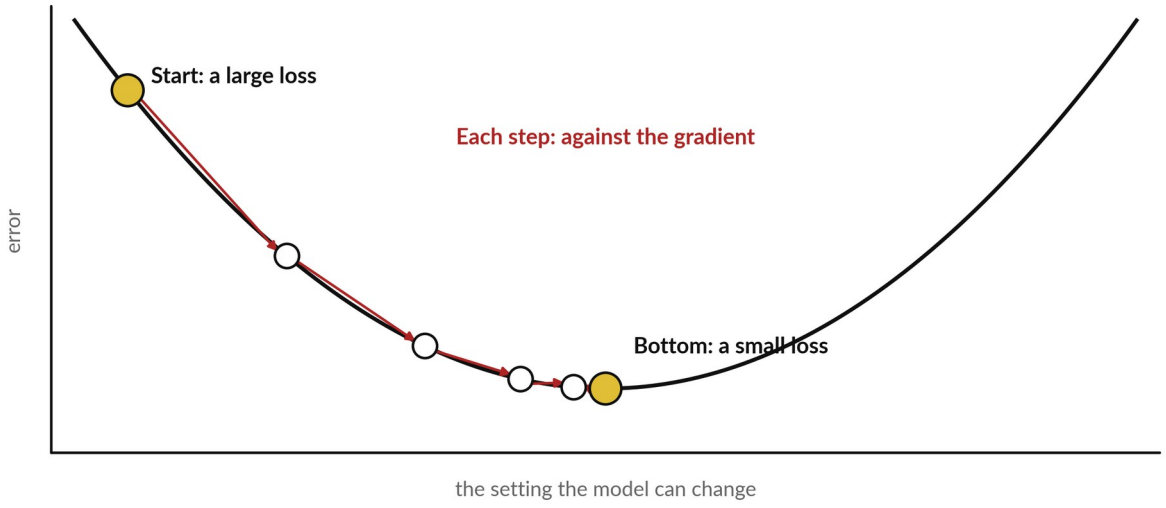
Training follows one loop, repeated many times. The parameters start as random numbers. The model predicts, and the loss is measured. The loss (also called the loss function, or the error) is one number that says how wrong the predictions are, and training tries to make it small. For house prices a natural loss is the average size of the price errors. Then the parameters are updated a little, and the loop repeats. One full pass over the training data is an epoch.




*The training loop. A smaller loss means better predictions.*

## Gradient descent

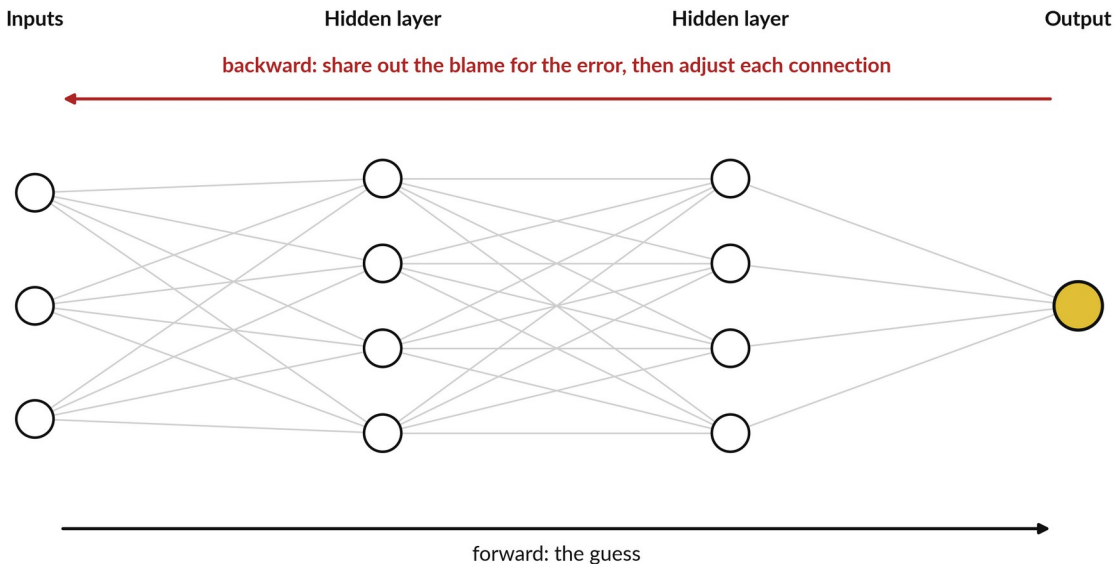
The gradient says, for each weight, in which direction and how steeply the loss changes when that weight changes. Gradient descent changes every weight by a small step against the gradient, so the loss falls. The learning rate is the size of each step: too big and the steps overshoot, too small and training is slow. As an analogy, think of walking downhill in thick fog: you cannot see the valley, but you feel the slope under your feet and step downhill. The analogy breaks in one place. The real "hill" has a direction for every weight, millions of them, and the walk can settle in a local minimum that is above the lowest point.




*Gradient descent. At the bottom the loss is the smallest the model has found. Simplified: the lowest point found is usually a good one rather than the best possible.*

## Backpropagation

A neural network has layers, and a prediction passes forward through all of them: the forward pass. The loss is measured at the output. Backpropagation then works out the gradient for every weight, layer by layer from the output backwards, using the chain rule of calculus. Gradient descent uses these gradients to update every weight. Together they train almost every modern neural network.




*Forward the network predicts and the loss is measured. Backward, backpropagation works out each weight's gradient, and every connection is adjusted.*

## Why training needs data and computing power

The model improves only by seeing many examples, so it needs a lot of data. Each prediction, loss measurement and update is a calculation, and a large model needs a huge number of them. That takes powerful specialised computers running for days or weeks. This is why only a few organisations train the largest models, and why most companies use a model someone else trained.

## Training, validation and test sets

You do not judge a model on the examples it studied. You split the data into three sets.

| Set | What it is for |
|---|---|
| **Training set** | The examples the model learns its parameters from. Most of the data goes here. |
| **Validation set** | Examples set aside while building. You use them to compare models and tune settings such as the learning rate. They are not used for learning. |
| **Test set** | A locked set, used once at the end for an honest score on cases the model has never met. Nobody tunes the model on it, because then it would no longer be a fair check. |

## Overfitting

Overfitting is when a model fits its training data, including its noise, and does worse on new data. Plainly put, the model has memorised its examples. Everyday example: a spam filter learns from 2,000 example emails, its training set. If it memorises those exact emails, word for word, it can score 100 percent on them. Then new spam arrives with different words, and the filter fails on it. Suppose it scores 80 percent on a test set of new emails (invented numbers). The big gap between 100 and 80 is the warning sign.

This is why a model that is 99 percent right on its training data can still fail on new data. The 99 percent only says it remembers what it saw. The test set answers the real question: how does it do on cases it has not seen? More varied data, a simpler model, or stopping training earlier all help. Keep this in mind when anyone shows you a very high accuracy figure: ask which data it was measured on.

Part 6

# Data: the fuel and the risk

Every AI system depends on data. Knowing what kind of data you hold, and how good it is, decides what AI can do for you.

## Kinds of data

| Kind | What it looks like | In a company |
|---|---|---|
| **Structured** | Fits neatly in rows and columns with fixed fields. | A sales sheet, an invoice register, a stock list |
| **Semi-structured** | Has some labels or layout but no fixed table. | Emails with sender, date and subject; forms; many exported files |
| **Unstructured** | Free-form content without a table. | Photos of damaged goods, voice recordings of calls, contracts, WhatsApp messages |
| **Time series** | Values recorded in time order. The order matters. | Daily sales, truck fuel readings, temperature sensor readings in a cold store |

Data can also be labelled or unlabelled. Labelled data carries the right answer: an email marked "complaint", an invoice marked "wrong total". Unlabelled data has no answer attached. Labelled data is needed for supervised learning and is usually costly, because a person has to do the marking.

## Data quality dimensions




A tool fed with poor data gives poor answers, quickly and with confidence. A simple example of poor consistency: sales data where half the shops write "Mbale" and half write "MBL" will be counted as two places.

## Bias and where it enters

A model reflects the data it learned from. Bias means a systematic tilt that treats some cases or groups unfairly. It can enter at four points:

- **In the data collected.** If past hiring favoured one kind of candidate, a model trained on it will too.
- **In the labels.** If people marked past cases unfairly, the model copies the marks.
- **In who is missing.** If few rural customers are in the data, the model serves them badly.
- **In how the result is used.** A sensible score used for a purpose it was not built for can still do harm.

## Privacy and the law in Uganda

Data about identifiable people (names, phone numbers, ID numbers, health or pay details) is personal data. In my understanding, the Data Protection and Privacy Act 2019 applies to personal data, including when it goes into an

AI tool; confirm your company's duties with a lawyer. Sessions 13 and 14 turn this into practical rules for your company.

Part 7

# Language models and generative AI

The assistants you use, such as ChatGPT, Claude, Gemini and Copilot, are built on large language models (LLMs). A language model is a model trained on a very large amount of text to predict what comes next.

## Words as numbers: vectors and embeddings

A computer handles only numbers, so text must become numbers. A vector is a list of numbers, and each position in the list is a dimension. Everyday example: three friends rate two films out of 10. Ana rates Film 1 a 3 and Film 2 a 4, so Ana is the vector (3, 4). Ben is (6, 8) and Chloe is (4, 3). Who has a taste closest to Ana? It depends on the measure. Euclidean distance is the straight-line gap: Ana to Ben is the square root of (3 - 6)^2 + (4 - 8)^2 = the square root of 25 = 5, and Ana to Chloe is 1.4. Manhattan distance adds the gaps along each axis: |3 - 6| + |4 - 8| = 7 for Ben and 2 for Chloe. The dot product multiplies matching ratings and adds them: 3 x 6 + 4 x 8 = 50 for Ben and 3 x 4 + 4 x 3 = 24 for Chloe. Cosine similarity compares only the angle between the two vectors: Ben points exactly the same way as Ana (1.00) and Chloe is slightly off (0.96). So the two distances pick Chloe, and the dot product and cosine similarity pick Ben. Know which measure a tool uses.

An embedding is a vector, produced by a trained model, that represents the meaning of a text, so that texts with similar meaning sit close together. A real embedding has hundreds of dimensions, and nobody reads the numbers: tools compare them, usually with cosine similarity. Older methods are simpler. A bag of words counts how often each word appears in a message and loses the word order. TF-IDF (term frequency times inverse document frequency) is a count in which rare terms weigh more than common ones.

## Tokens and next-token prediction

Your phone keyboard suggests the next word as you type. A language model does the same thing at far greater scale. It does not read whole words. A tokenizer splits text into tokens: short words, parts of long words or punctuation marks, taken from a fixed list called the vocabulary (50,257 tokens in GPT-2, an open model). The model gives a score, called a logit, to every token in its vocabulary. Softmax turns the logits into probabilities that add up to 100 percent. For example, logits of 2, 1 and 0 become about 67, 24 and 9 percent. The formula in words: the probability of a token is e raised to its logit, divided by the total of those values for all tokens, where e is about 2.72. The model picks one token, adds it to the text, and repeats with the longer text. This is next-token prediction, and an answer of a hundred words is built one token at a time.

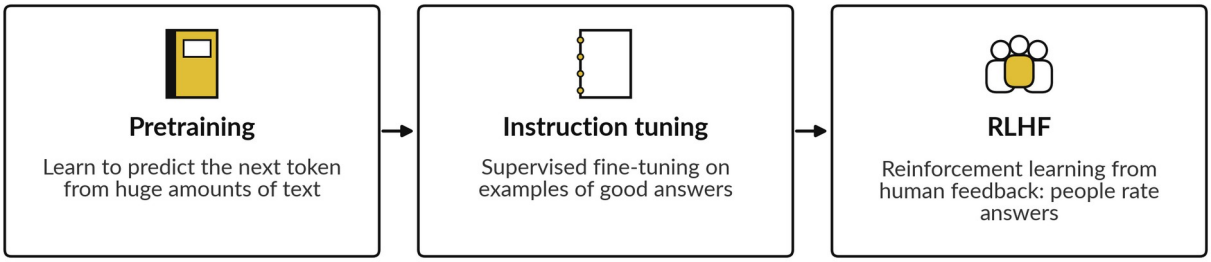
## Temperature and sampling

Choosing a token from the probabilities is called sampling. Temperature is a setting that divides the logits before softmax: a low temperature sharpens the probabilities so the top token wins more often, and a high temperature flattens them so answers vary more. Top-k sampling keeps only the k most likely tokens. Top-p sampling keeps the smallest group of tokens whose probabilities add up to p. Greedy decoding always takes the single top token. This is why the same question can give different answers.

## Self-attention and the transformer

To predict well, each token must take account of the other tokens. Self-attention does this: each token weighs the earlier tokens by how relevant they are. Simplified: each token makes a query (what it is looking for), each earlier token offers a key (what it offers, to be matched) and a value (the information it hands over). How well a query matches a key sets how much of that token's value is taken. An attention head is one set of queries, keys and values. A transformer block is self-attention followed by a feed-forward network, and a transformer, the architecture of modern language models, is a stack of such blocks (GPT-2 small has 12). Before the first block, each token is turned into a row of numbers taken from the embedding matrix, a lookup table with one row per token, and positional encoding adds numbers that tell the model where the token sits.

## How a language model gets built




*Pretraining gives a base model, which continues text. The next two stages are post-training and give an instruction-tuned model. Simplified: companies combine several methods.*

## Context window

The context window is how many tokens the model can keep in view at once: your prompt, any files you give it, and its own answer so far. As an analogy, think of the desk the model works on. If a document is larger than the desk, parts fall off, and the model may miss or mix them up. Windows are now large, but a bigger desk does not guarantee that the model attends to every part equally well.

## Why a model can sound sure and be wrong

The model is built to produce likely text that reads well. It is not built to check facts. When it lacks the information, it can still write a smooth, confident sentence that is false. This is called a hallucination. It can include invented figures, names, quotes and sources. The tone does not tell you whether it is right. This is why the course teaches you to ask for sources, to check, and to test a tool on your own work (Session 10).

## What changes with search, files and memory

On its own, a model answers from what it learned in training, which stops at a date and knows nothing about your company. Assistants can add three things.

- **Search.** The assistant looks up the web and writes from what it found. You can open the sources.
- **Files.** You give it a document and it answers from that document.
- **Memory.** Some assistants keep notes about you between chats. Check what is kept and who can see it.

Giving the model the right passages from your own documents before it answers is called retrieval-augmented generation (RAG). A retriever finds the relevant passages first and the model writes from them, like an open-book answer. This cuts down mistakes but does not remove them: the retriever can fetch the wrong passage, and the model can still misread the right one. Always check the answer against the source.

Part 8

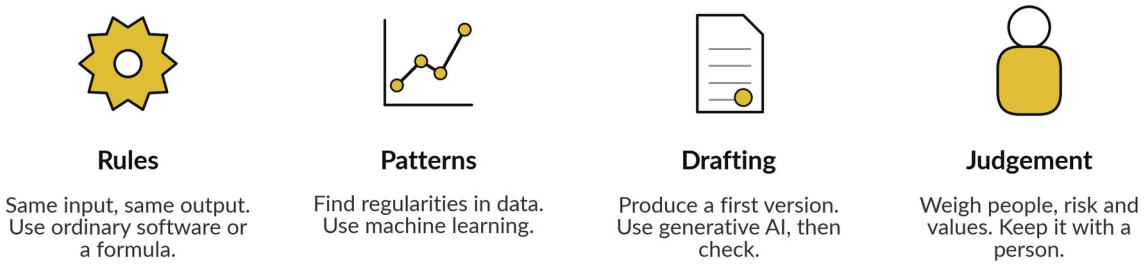
# Where AI is used in industry

AI is used wherever there is a lot of data, a repeated decision and a cost to getting it wrong or slow. These are general examples of where it is used. They contain no statistics.

| Sector | Typical uses |
|---|---|
| **Banking and mobile money** | Spotting unusual transactions that may be fraud; estimating the risk that a borrower will not repay; answering customer questions. |
| **Agriculture** | Recognising crop diseases and pests from photos; forecasting yield and market prices; advising farmers by message. |
| **Retail and distribution** | Forecasting demand for each product and shop; planning delivery routes; setting reorder points. |
| **Health administration** | Scheduling and reminders; sorting and coding records; drafting letters. Clinical decisions stay with clinicians. |
| **Human resources** | Sorting applications and drafting job adverts. Screening carries a bias risk, so a person must review the outcome. |
| **Customer service** | Sorting incoming messages, drafting replies, finding the right policy page for the agent. |
| **Finance and accounting** | Matching invoices to purchase orders; flagging duplicates and wrong totals; drafting management commentary. |

## What suits AI and what does not

Work comes in four kinds. Knowing which kind you have tells you which tool to reach for.




A good first task for AI is one with a clear pattern or a draft to produce, where a person can check the result quickly and a mistake is cheap to catch. A poor first task has a high cost of error, little data and nobody to check. Session 12 gives you a scorecard for this choice. A rule is often better done with a formula, which is cheaper, faster and always the same.

## The fields of AI

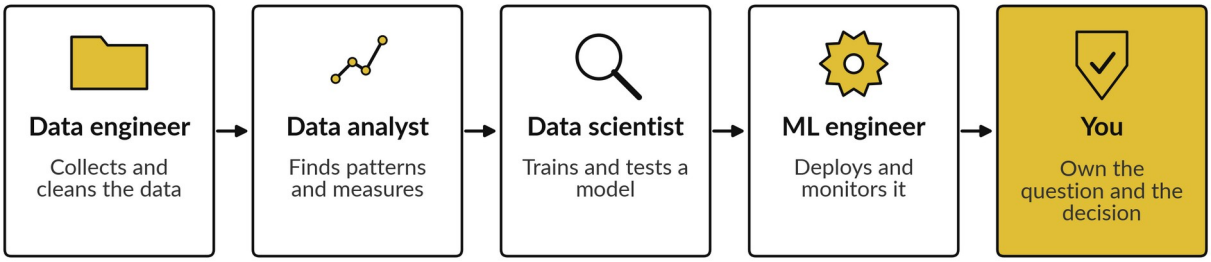
AI is not one skill. It is a set of fields, and each one suits a different business problem. A vendor who says they do AI should be able to say which field they work in.

| Field | What it does | In a company |
|---|---|---|
| **Natural language processing (NLP)** | Reads and writes human language: summarising, sorting, translating, answering. | Summarise emails, sort enquiries, draft replies. |
| **Computer vision** | Understands images and video. | Read invoices, check goods in photos, count stock. |
| **Speech** | Turns speech into text and text into speech. | Transcribe meetings and calls. |
| **Forecasting (time series)** | Predicts numbers over time from past values. | Demand, cash flow, prices. |
| **Recommendation** | Suggests what a person may want next. | Product suggestions, next best offer. |
| **Anomaly detection** | Spots what does not fit the usual pattern. | Fraud alerts, faulty machines. |
| **Deep learning** | A family of methods using large neural networks, behind most progress in language, vision and speech. | Inside most of the tools above. |
| **Generative AI** | Creates new text, images, audio or code. | Draft reports, replies and code. |
| **Reinforcement learning** | Learns by trial and reward. | Route planning, dynamic pricing, robots. |

## The jobs behind AI work

Titles vary between companies, and in a small company one person often does several of these jobs. This is the usual division.

| Role | What they do | Where you meet it |
|---|---|---|
| **Data engineer** | Builds and runs the pipes that bring clean data in. | Session 14 |
| **Data analyst** | Explains what happened, with tables and charts. | Session 12 |
| **Data scientist** | Builds and tests models that predict or classify. | Sessions 2 and 3 |
| **Machine learning engineer** | Turns a model into a reliable product; deploys and monitors it. | Sessions 7 and 11 |
| **MLOps engineer** | Builds the systems that train, release and monitor models. | Session 7 |
| **Applied AI engineer** | Builds products from ready-made models with prompts, retrieval, automations and agents. | Sessions 8 to 11 |
| **AI researcher** | Invents new methods, usually in universities or large labs. | Not needed to start |
| **Data steward or information manager** | Owns the quality and the rights of the data: who may use it, how it is kept and how it is corrected. | Session 14 |
| **Product owner or AI governance lead** | Chooses use cases, sets the rules and checks the value. | Sessions 12 to 14: your seat |




*The roles hand work along a line. At every step a leader owns the question, the rules and the decision. If your company has none of these people, you can still do useful work with ready-made tools, which is what the practicals show.*

Part 9

# How we know a model is good

Every vendor quotes a number: "95 percent accurate", "an AUC of 0.9". This part gives you the words to read those numbers and the questions to ask about them. All figures in this part are illustrative.

## Accuracy, and why it can mislead

Accuracy is the share of all cases the model got right. It sounds like the whole story, and for rare events it is not. Everyday example: a spam filter checks 1,000 emails, and 50 of them are really spam. A majority-class baseline, a simple reference model that always predicts the most common class (here, "not spam"), is right 950 times in 1,000 and catches no spam at all. A high accuracy can hide a useless model, so we look at the kinds of right and wrong.

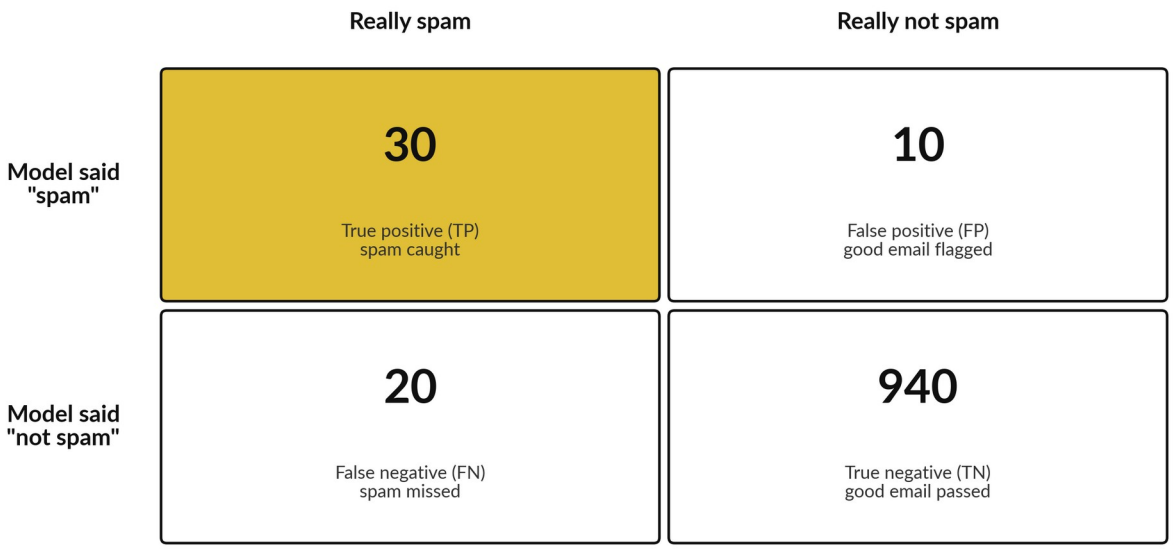
## The confusion matrix

For a model that sorts cases into two groups, such as spam and not spam, every case lands in one of four outcomes. The table that counts them is the confusion matrix.

| Outcome | What it means |
|---|---|
| **True positive (TP)** | The filter says spam, and it is spam. A catch. |
| **False positive (FP)** | The filter says spam, and it is a good email. A false alarm. |
| **False negative (FN)** | The filter says not spam, and it is spam. A miss. |
| **True negative (TN)** | The filter says not spam, and it is a good email. A correct pass. |

## One worked example (illustrative)

The filter flags 40 of the 1,000 emails as spam. Of those 40, 30 are really spam (TP) and 10 are good emails (FP). It misses the other 20 spam emails (FN). The remaining 940 good emails are correctly passed (TN). These numbers are made up to show the arithmetic.




*Illustrative confusion matrix for a spam filter on 1,000 emails.*

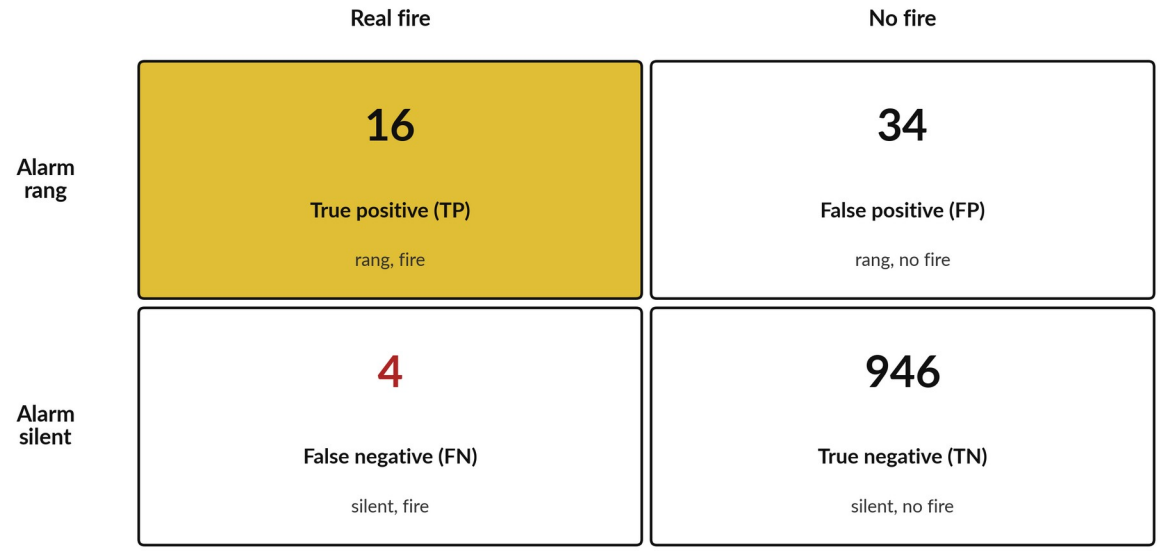
| Measure | The question it answers | Arithmetic | Result |
|---|---|---|---|
| **Accuracy** | Right answers out of all cases | (30 + 940) / 1,000 = 970 / 1,000 | 97 percent |
| **Precision** | When it says spam, how often is it right? | 30 / (30 + 10) = 30 / 40 | 75 percent |

| Measure | The question it answers | Arithmetic | Result |
|---|---|---|---|
| **Recall** | Of the real spam, how much did it catch? | 30 / (30 + 20) = 30 / 50 | 60 percent |
| **F1 score** | One number that balances precision and recall | 2 x 0.75 x 0.60 / (0.75 + 0.60) = 0.90 / 1.35 | about 67 percent |

The accuracy of 97 percent looks excellent, yet the baseline that never flags anything scores 95 percent. The filter catches only 60 percent of the spam, and 10 good emails are lost. Precision protects the good emails. Recall protects you from spam. Which matters more depends on what a false alarm and a miss cost you. The F1 score is the harmonic mean of precision and recall, so it stays near the weaker of the two: precision 90 percent with recall 10 percent has an average of 50 but an F1 of only 18.

## The threshold, shown by a smoke alarm

Most models give a score. The threshold is the score at or above which the model says positive, and you choose it. Everyday example: a smoke alarm rings when the smoke level passes its threshold. In 1,000 kitchen events, 20 are real fires. Our alarm rings 50 times: 16 are fires (TP) and 34 are false alarms (FP). It stays silent for 4 real fires (FN) and for 946 harmless events (TN). Precision is 16 / 50 = 32 percent, recall is 16 / 20 = 80 percent, accuracy is 962 / 1,000 = 96.2 percent, and F1 is 2 x 0.32 x 0.80 / 1.12 = 46 percent. An alarm that never rings is 98 percent accurate and catches no fire, so it is the baseline to beat on recall.



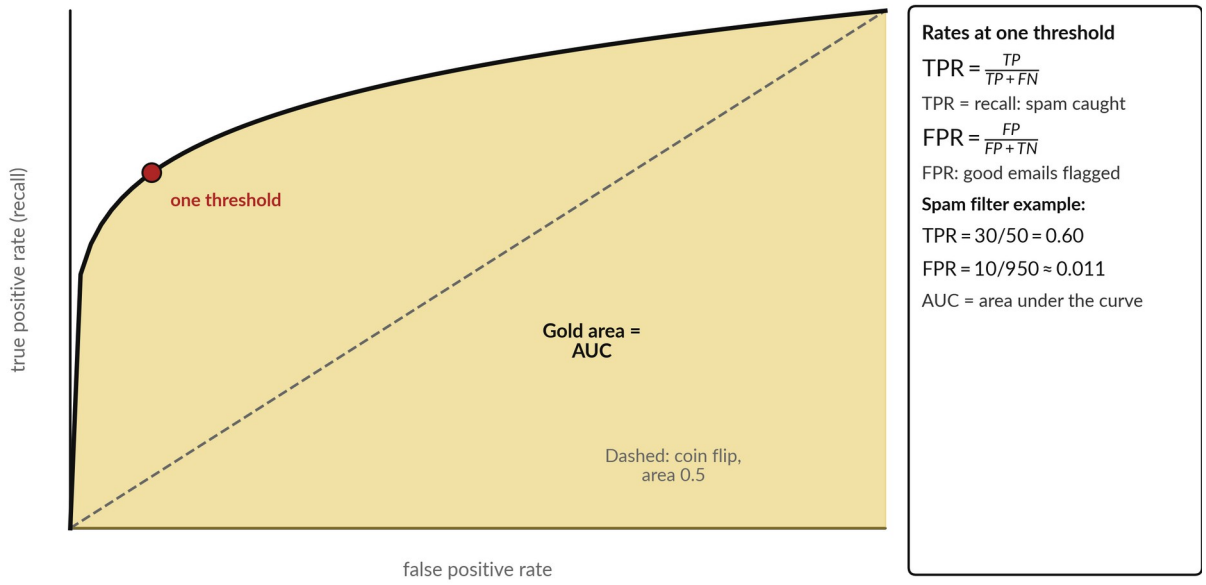

*The smoke alarm confusion matrix (invented numbers): 1,000 kitchen events, 20 real fires.*

| Threshold | Fires caught | False alarms |
|---|---|---|
| **High (rings late)** | 12 of 20 | 8 |
| **Ours** | 16 of 20 | 34 |
| **Low (rings early)** | 19 of 20 | 181 |

Set the threshold low and you catch more fires (recall rises) but raise more false alarms (precision falls). Set it high and the reverse happens. The threshold is a business decision about cost, and the data scientist should not choose it alone.

## The ROC curve and AUC

The ROC curve (receiver operating characteristic) shows what happens as you slide the threshold. It plots the share of real positives caught (the true positive rate, which is recall) against the share of negatives wrongly flagged (the false positive rate). A curve that bows toward the top left corner is a better model. The area under the curve is called AUC. An AUC of 0.5 is a coin flip and 1.0 is perfect. In the illustration below the AUC is about 0.86. One way to read it: an AUC of 0.80 means a randomly chosen positive case gets a higher score than a randomly chosen negative case 80 times in 100. When positives are very rare, also check precision.




*Illustrative ROC curve for the spam filter. The dashed diagonal is a coin flip. The further the curve bows toward the top left, the better the model.*

## Cross-validation: an honest score from limited data

When you have little data, a single split into training and test sets can flatter or punish a model by luck. Cross-validation (k-fold) fixes this. Split the data into k parts, called folds. Train on k - 1 folds and test on the one held-out fold, then rotate so that each fold is the test fold once, and average the k scores. With 40 messages and k = 5, each fold holds 8 messages: train on 32, test on 8, five times. Session 2 shows it.

## Measures for models that give a number

| Measure | In plain words |
|---|---|
| **MAE (mean absolute error)** | On average, how far off the prediction is, in the same unit as the thing predicted. If the MAE of a house price model is UGX 20 million, a predicted price is typically 20 million away from the real one. |
| **RMSE (root mean squared error)** | Like the MAE, but a big miss counts for much more than a small one. Use it when large errors hurt most. |
| **R squared** | How much of the ups and downs in the real numbers the model explains. 1 means all of it, 0 means none. |

## Measures for ranking and recommendation

When a system suggests a list, such as films to watch next, a simple measure is the top-k hit rate: in how many cases was what the person really wanted among the first k suggestions? With k = 5, a hit rate of 60 percent would mean that in 6 cases out of 10 the right film was in the top five.

## Measures for language models

A language model writes free text, so there is no single right answer to count. Five checks are in common use.

| Check | The question it answers |
|---|---|
| **Benchmarks** | Public sets of standard questions (see the next part). Useful for a first comparison. |
| **Human review** | People read the answers and rate them against a short checklist. Slow, and the most trusted. |
| **Groundedness** | Does every claim in the answer come from the documents the model was given? |
| **Citation accuracy** | Does each cited passage exist, and does it say what the answer says it says? |
| **Hallucination rate** | How often does the answer contain something invented? |

For your own work, build a small test set from your own documents: a few dozen real questions with answers you have checked. Run the tool on them and count the results. No public benchmark can tell you how a tool does on your contracts, your reports and your words.

**Questions to ask about any quoted figure**

1. Measured on which data? Was it data the model had never seen, and does it look like ours?
2. At which threshold? A figure at one threshold says little about another.
3. What does a miss cost us, and what does a false alarm cost us?
4. Compared with which baseline? How does the task get done today, or by a very simple method?

Part 10

# Benchmarks

A benchmark is a set of standard test questions given to many models, so that their scores can be compared. You will see them in vendor announcements.

- **MMLU** tests broad knowledge across many school and professional subjects, by multiple-choice questions.
- **GSM8K** tests grade-school maths word problems.
- **HumanEval** tests coding: the model writes small programs, and the programs are run to see whether they work.
- **Chatbot Arena** a public site where people compare two anonymous answers and vote for the better one. The votes build a ranking of human preference.

## Limits of benchmarks

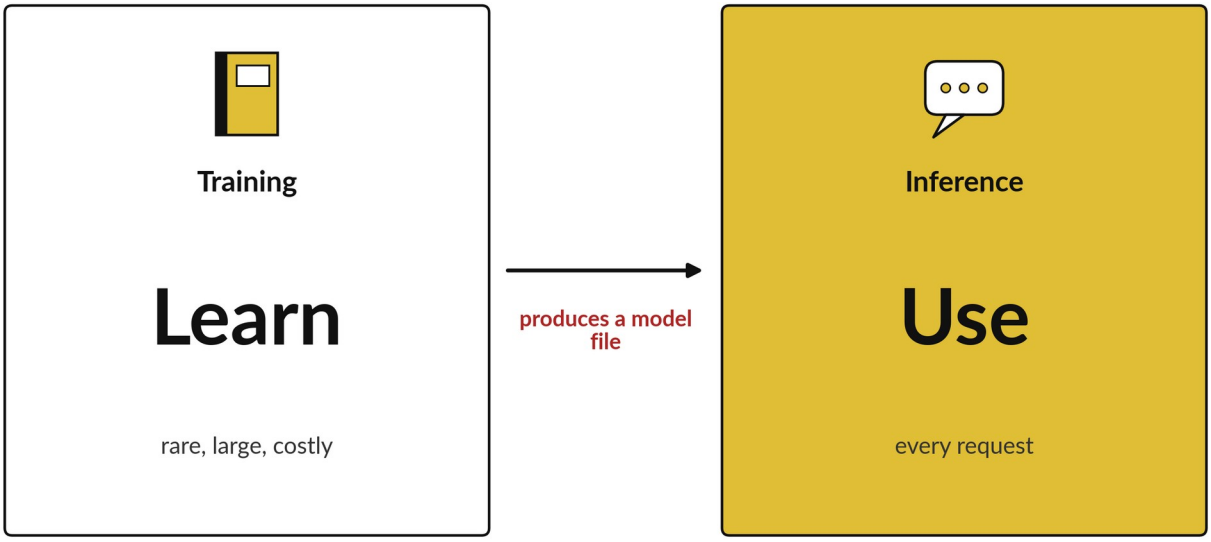
- **Contamination.** If the test questions were in the text the model learned from, the model may have seen them. The score then overstates what it can do.
- **Who ran it.** A vendor-run score deserves less weight than an independent one. Ask who ran the test and whether others can repeat it.
- **Your task is different.** A model that is strong on school-style questions may be weak on your contracts, your language or your forms.

The maker also publishes a model card: a page that states the model's licence, size, intended uses and limits. Use benchmarks and the model card to shortlist two or three candidates. Then test those candidates on your own documents, with your own questions, before you decide.

Part 11

# Training and inference

Two words describe the two phases of a model's life. Costs and skills differ in each.




*Training makes the model file once. Inference uses the file again and again.*

| | Training | Inference |
|---|---|---|
| **What happens** | The model learns its parameters (weights and biases) from data. | The finished model answers a new request. |
| **How often** | Rarely: once, and again when the model is updated. | Every time someone asks. |
| **What it needs** | Large data and a great deal of computing power. | Fast, reliable serving, so that answers come back quickly. |
| **What it produces** | A model file. | An answer. |
| **Speed** | Takes days or weeks, but nobody waits for the answer. | Latency is the wait for an answer. Throughput is how many requests are served per second. |
| **Where the cost sits** | A large one-off cost, paid by the model maker. | Billed per use, which is the cost you pay and which grows with your volume. |

Part 12

# What happens when you press send

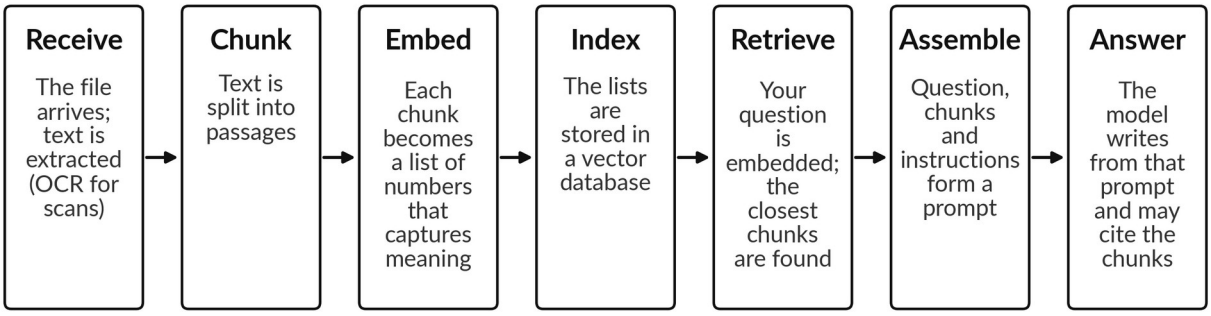
You type a question and press send. An answer appears in seconds. This part opens the box. The steps are simplified, and each product differs in the details.

## A plain message

- **1. Split.** Your text is cut into tokens.
- **2. Predict.** The model predicts the most likely next token, adds it, and predicts again, over and over, until the answer is complete.
- **3. Check and send.** The reply is checked by the provider's safety rules and streamed back to you as it is produced, which is why the words appear one after another.

### A message with a document

When you also upload a document, more happens before the model writes anything. This is the pipeline behind a "chat with your documents" tool.




*The seven stages of a retrieval pipeline, read left to right. The first four happen when the file is added. The last three happen on every question.*

An embedding is a list of numbers that captures the meaning of a passage, so that passages with similar meaning sit close together. A vector database stores these lists and finds the closest ones to a question quickly. Products differ: a short file may simply be placed whole in the context window, and only long files go through retrieval. The exact pipeline is the vendor's, so ask.

| Stage | What it does | What can go wrong |
|---|---|---|
| **Receive and extract** | Reads the file and pulls out the text. A scan needs optical character recognition (OCR). | A scan is misread: a 3 becomes an 8, a stamp covers a line, a table loses its columns. |
| **Chunk** | Cuts the text into passages of a workable size. | A clause is split across two chunks, so neither chunk holds the full meaning. |
| **Embed** | Turns each chunk into numbers that capture meaning. | Rare terms, names and codes are captured poorly. |
| **Index** | Stores the embeddings so they can be searched. | A stale index: the document changed but the index did not. Permissions are ignored, so a person sees a passage they should not. |
| **Retrieve** | Finds the chunks closest in meaning to the question. | The wrong chunk is retrieved, and the model answers confidently from the wrong page. |
| **Assemble and answer** | Builds the prompt and generates the answer, possibly with citations. | The model misreads the right chunk, or a citation points to a passage that does not say that. |

Part 13

# Foundation models, open models, fine-tuning and cost

A foundation model is trained once on broad data and then adapted for many tasks. Few companies train one. Most use one. The questions for you are which kind to use, how far to adapt it, and what it will cost to run.

## Proprietary models and open weights

A proprietary model is reached by subscription or through an API, and the maker keeps the model file. A model with open weights can be downloaded and run on your own machines. Be careful with the words "open source": under the Open Source Initiative (OSI) definition open source needs more than the weights, and the licence of an open-weights model may still limit how you use it, so read the licence.

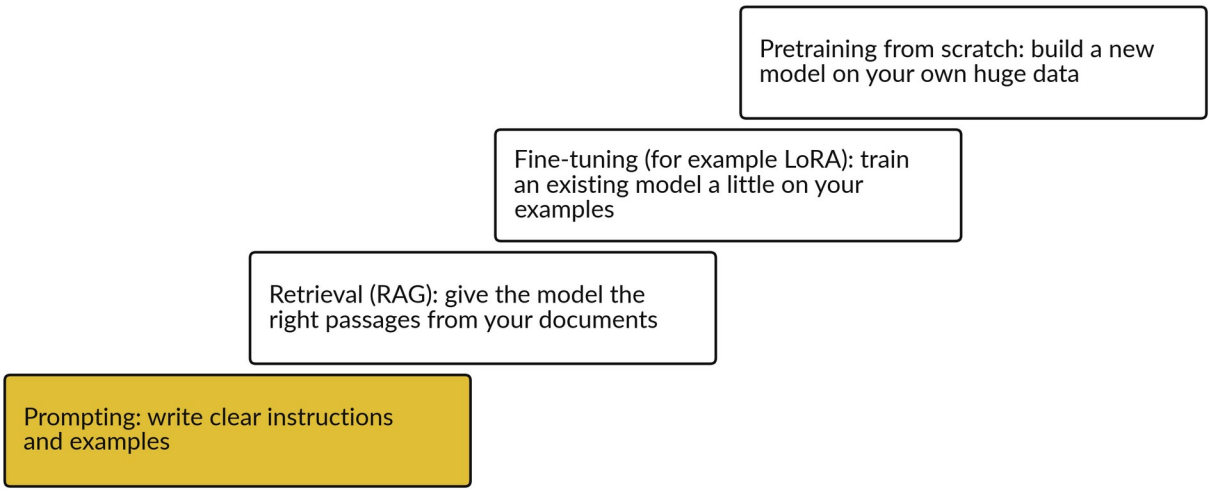
| | Proprietary | Open weights |
|---|---|---|
| **Set-up effort** | Low: sign up and start. | Higher: you set up the servers and the software. |
| **Control of data** | Your data goes to the provider. Check the terms and where it is processed. | Data can stay on your own machines. |
| **Quality** | Often the strongest general models. | Improving fast; strong for many tasks, a gap on some. |
| **Skills needed** | Little technical skill. | Engineers who can run and secure the model. |
| **Running cost (kind)** | Pay per use or per user. | Computers and people, whether or not the model is busy. |
| **Who maintains it** | The provider. | You, or a partner you hire. |

## Size, precision and quantisation

A model file is a list of parameters, so its size is a counting problem: the number of parameters times the bits stored for each. Precision is the number of bits per parameter: FP32 is 32-bit floating point and FP16 is 16-bit. Quantisation lowers the bits per parameter, for example to 8-bit integers (INT8) or 4-bit, so that the file is smaller and runs on cheaper hardware, usually with a small loss of quality. A file format such as safetensors, GGUF or ONNX is only the container that packs the parameters, and it says nothing about the precision. Session 7 works the memory sum.

## Four ways to adapt a model to your task

Fine-tuning, for example with LoRA (low-rank adaptation, which trains a small adapter and leaves the original parameters unchanged), teaches style and skills. Retrieval supplies facts. Most business needs start with prompting and retrieval.



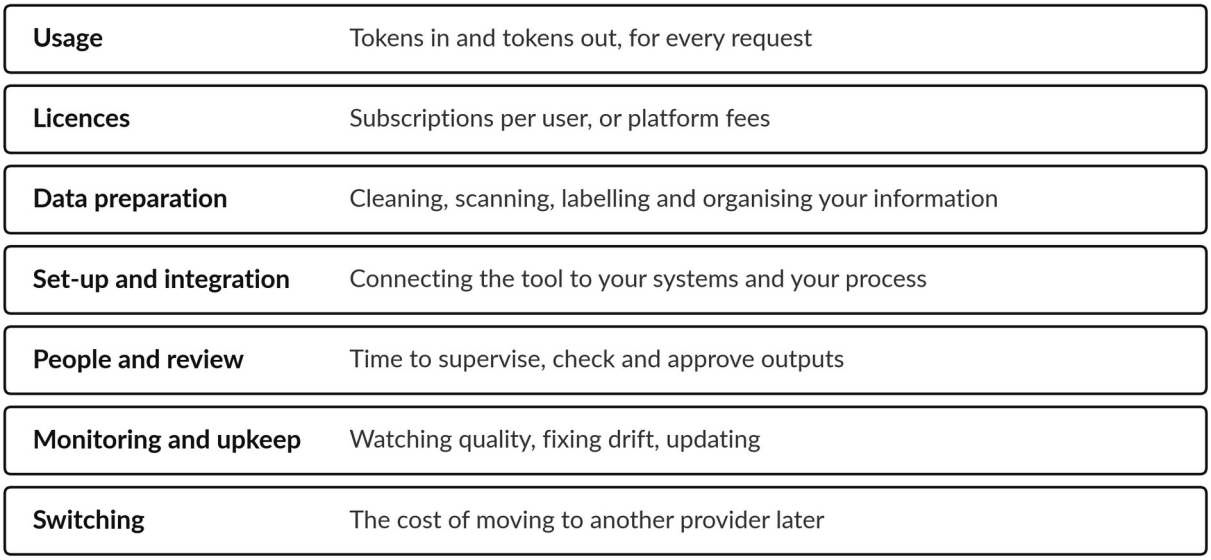

*Start at the bottom. Move up only when the step below cannot do the job.*

| Method | What it changes | When to use it | Effort |
|---|---|---|---|
| **Prompting** | The instructions the model sees. | Always the first step. | Hours |

| Method | What it changes | When to use it | Effort |
|---|---|---|---|
| **Retrieval-augmented generation (RAG)** | The facts the model can see when it answers. | The answer must come from your own, changing documents. | Days to weeks |
| **Fine-tuning (for example LoRA)** | The model's parameters, a little: its style, format and skills. | You need a steady tone or format that prompts cannot give, and you have good examples. | Weeks, with specialists |
| **Pretraining from scratch** | Everything. | Almost never for a company. | Months, large teams and data |

## Where AI costs come from

The model fee is one of several costs. A business case that counts only the fee will come out too low. This stack shows the parts. No figures are given, because each depends on your plan and your volume.




*Seven kinds of cost. Session 12 puts them into your business case.*

Part 14

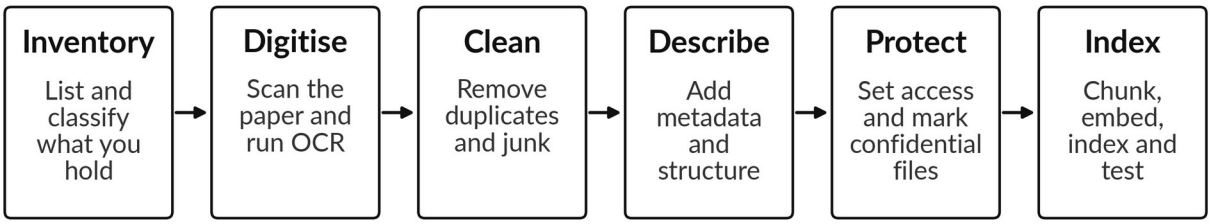
# Is our data ready for AI?

A tool is only as good as the information behind it. Consider Nile Valley Foods, which wants an assistant that answers questions across its own records: a signed supplier contract, a year of invoices and a set of HR files. They exist as paper in a filing cabinet, scanned PDFs of mixed quality on a shared drive, Word documents, emails, and a spreadsheet index that someone started years ago. The company wants answers in seconds. Can it do that today? The scorecard below shows how to find out.

## The readiness scorecard

| Check | The question | The red flag |
|---|---|---|
| **Data inventory** | Do we know what we hold, and where? | Nobody can say how many documents there are, or where they sit. |
| **Readability and accuracy** | Can a machine read it? | Crooked, faint or handwritten scans; text that OCR reads wrongly. |
| **Completeness and duplicates** | Is each document present once? | Five copies of one contract in different folders, or pages missing. |
| **Metadata** | Is each file labelled with type, date, owner, parties and reference number? | File names such as "scan0042.pdf" and no labels at all. |
| **Rights and confidentiality** | May we use it, and who may see it? Are personal and confidential records protected? | Confidential HR files mixed in with documents anyone may see. |
| **Timeliness** | Is it up to date? Are superseded versions marked? | An expired contract or old policy is returned as current. |
| **Structure** | Does it have headings, sections and paragraphs we can rely on? | One long block of text with no divisions. |
| **Access and ownership** | Who owns it, and who can open it? | Everyone can open everything, or nobody is responsible for it. |

## Six preparation steps




*Six steps, left to right, from a filing cabinet to a searchable collection. Step 6 is the pipeline from Part 12, taught in Session 9.*

## How to test the result

Write about 50 real questions to which you already know the answers, drawn from the work your teams really do, across contracts, invoices and HR. Run them through the tool and record four things.

- **Retrieval.** Was the right passage found?
- **Faithfulness.** Does the answer say what the passage says, and nothing more?
- **Citations.** Does each citation point to the right document and the right place in it?
- **Time saved.** How long did it take compared with the usual way?

Fix the weakest step, run the same questions again and compare. This test set also protects you later, because you can rerun it whenever the tool or the documents change.

## Preparing data to train or fine-tune a model

Preparing data to train or fine-tune a model is a different job from preparing it for retrieval. The steps are:

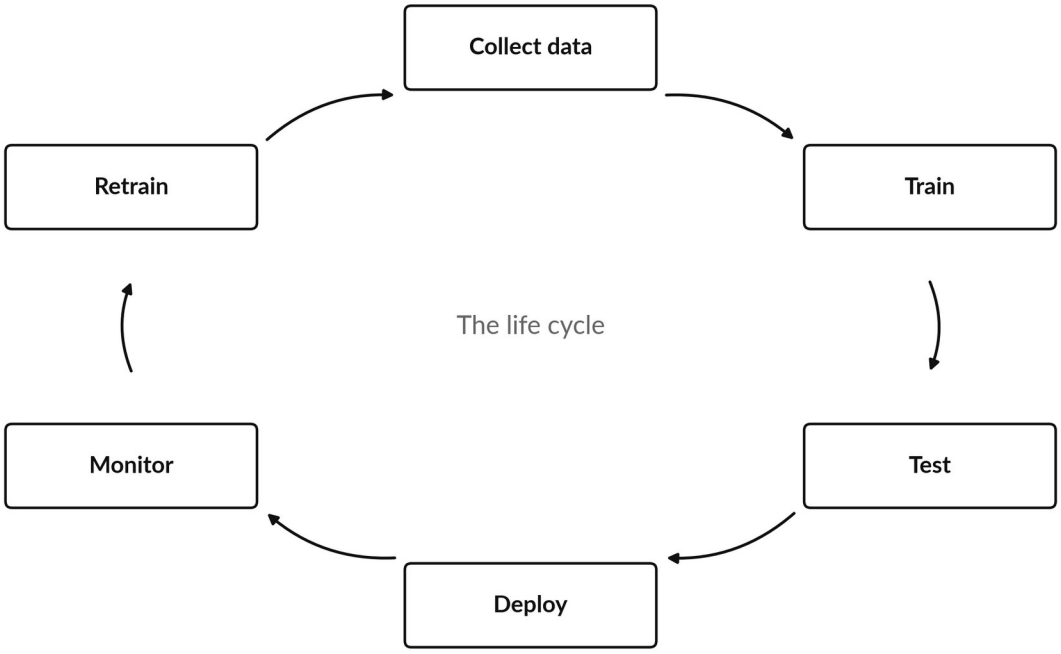
- Collect data that you have the right to use for this purpose.
- Clean it and remove personal data.
- Filter for quality: drop poor, wrong or repeated material.
- Format it as examples, such as question and answer pairs.
- Split it into training, validation and test sets.
- Tokenise it, which means cutting it into tokens the model reads.
- Train, then evaluate on the test set.

This asks for more data, more skill and more money, and the result is a model that has absorbed your documents in a way you cannot easily inspect or correct. For this reason most organisations should prepare for retrieval first. Retrieval keeps each fact tied to its source, lets you delete or update a document, and works with the model you already use.

Part 15

# From model to working product

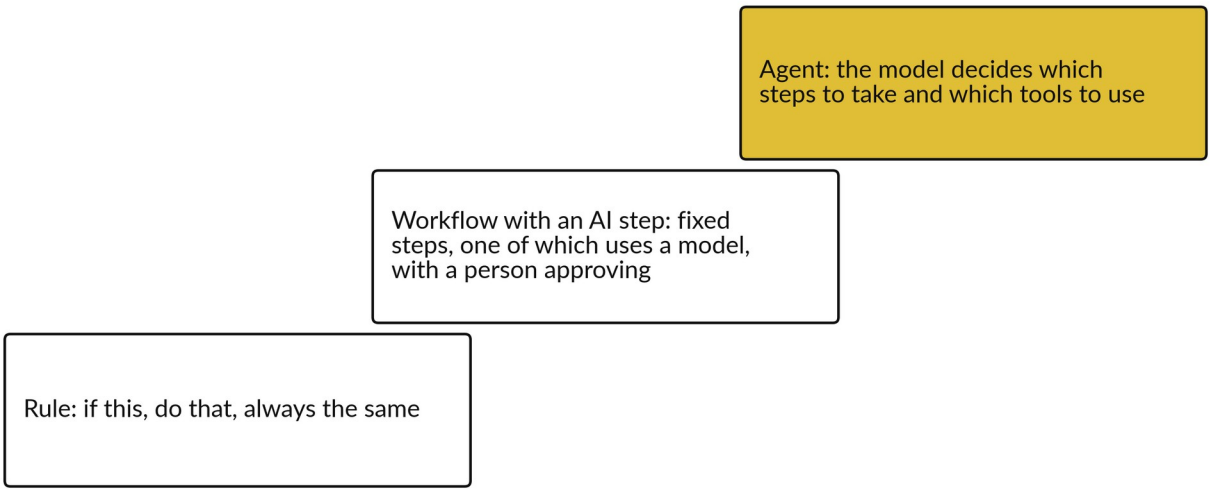
A model on its own does nothing for a company. It becomes useful when it sits inside a process that people use and look after. The life of a working AI system goes round in a cycle.




*Deploy means put it to work. Monitor means watch the results. Drift is when the world changes and the model slowly gets worse, which is why retraining is part of the cycle.*

## Models, workflows and agents

Three levels of automation are easy to mix up. Each gives the system more freedom, and so needs firmer limits.




*The ladder runs from the most predictable at the bottom to the most freedom at the top.*

Session 12 shows you the first two steps side by side on the same messages: a rule, and a workflow with an AI step and a person approving. Session 11 looks at agents. The more freedom a system has, the more it matters to set limits on what it may do and to keep a person approving anything that leaves the company, such as an email to a customer, a payment or a change to a record. Human-in-the-loop means an approval step: a person must approve, edit or reject the action before it runs.

## Agents: tools, permissions and guardrails

An agent is an AI model given a goal and tools, which chooses its own next step in a loop of plan, act and observe. Tool use (also called function calling) means the model writes a request to run a named tool, such as read_mail, and a program runs it and returns the result. The Model Context Protocol (MCP) is an open standard for connecting models to tools and data in a common way. Four ideas keep an agent safe.

- **Permissions (scopes).** What the agent may read, write, send and spend, written down in a permissions table. Session 11 gives you the Tool Boundary Sheet for this.
- **Least privilege.** Give the agent the smallest set of permissions that still lets it do the job.
- **Guardrails.** Checks and limits around the agent, such as spending limits, filters and an off switch, with an audit log that records each action.
- **Prompt injection.** Text hidden in an email, web page or file, written to make the agent follow orders its owner never gave. Treat everything the agent reads as untrusted.

## Automation tools on the market

Many tools can build the same workflow. They differ in who they suit and how they charge. The list below says how each is billed in kind. It gives no prices, which change and depend on your plan.

| Tool | What it suits | How it is billed (kind only) |
|---|---|---|
| **Zapier** | No-code connections between thousands of apps. Suits business users. Discussed as an option only. | Per task (a task is one completed action step) |
| **Make** | Visual canvas with branches and detailed control. Suits people who like to see the whole flow. | Per operation |
| **n8n** | Flexible and technical. Can be run on your own server. Suits teams with a technical person. | Per execution (a run of the whole workflow) on its cloud |
| **Microsoft Power Automate and Copilot Studio** | Best when your company already runs Microsoft 365. Connects to Outlook, Excel, Teams. | Per user, or by usage credits |
| **Agent builders in ChatGPT, Gemini and Claude** | Build an assistant or agent for your own instructions and files, inside the assistant you already use. | Part of the assistant plan, with limits that vary by plan |

Billing terms and plan limits change often. Check the current terms before you decide, and ask what happens to your data on each plan.

Part 16

# What to bring to Session 1

You have nothing to prepare before Session 1 except to read this handbook. A few small things make the demonstrations more useful.

## What to bring

- **A laptop and one AI assistant account.** Any assistant you already use will do, such as ChatGPT, Claude, Gemini or Copilot. You can join the demonstrations if you wish, and you can also simply watch. The course does not need a paid plan.
- **Use only sample or invented data.** In class and in practice, use made-up or sample information. The course gives you a fictional company, Nile Valley Foods Ltd, for this.

## Ground rules for responsible use




These three rules apply from the first session. The policy you draft in Session 13 builds on them.

**Questions to keep in mind from Session 1**

Where in my own work do I already meet a system that learns from examples?
Which AI claims have I been asked to trust without seeing a number?
What would "better" mean in my work, in a number I could measure?

Part 17

# Glossary

Terms in plain words, from A to Z.

| Term | In plain words | Session |
|---|---|---|
| **Accuracy** | The share of all cases a model got right. It can mislead when the event is rare. | 0, 3 |
| **Activation function** | A simple function applied to the weighted sum that adds non-linearity. ReLU (rectified linear unit) outputs 0 for negative values and the value itself otherwise; a step gives 1 or 0; sigmoid squeezes the value into 0 to 1. | 1 |
| **Agent** | An AI model given a goal and tools, which chooses its own next step in a loop of plan, act and observe. It needs firm limits. | 11 |
| **AI literacy** | Knowing enough about how AI works, where it fails and what rules apply, to use it and supervise it well. | 0 |
| **Algorithm** | A set of steps a computer follows to solve a problem or make a calculation. | 0 |
| **API** | Application programming interface: a door that lets one program talk to another. A tool uses an API to ask an AI model for a result. | 7, 11 |
| **AUC** | Area under the ROC curve: one number from 0.5 (a coin flip) to 1.0 (perfect). | 3 |
| **Automation** | Letting software do repeated steps without a person doing each one. | 12 |
| **Backpropagation** | The method that works out the gradient for every weight, layer by layer from the output backwards, so gradient descent can update them. | 0, 1 |
| **Baseline** | A simple reference model to beat, such as the majority-class baseline that always predicts the most common class. In a business case, how the task is done today. | 3, 12 |
| **Benchmark** | A fixed set of standard test questions with a score, used to compare models. Examples: MMLU, GSM8K, HumanEval. | 6 |
| **Bias (in a model)** | The number added to the weighted sum that shifts when a neuron switches on. Not to be confused with unfairness. | 1 |
| **Bias (unfairness)** | A systematic tilt in a model's results that treats some cases or groups unfairly, often copied from the data. | 10, 13 |
| **Chunk, chunking** | A passage of a document, cut to a workable size so that it can be embedded and retrieved. | 9 |
| **Classification** | Predicting a category, such as spam or not spam. | 2 |
| **Clustering** | Finding groups of similar cases without being told the groups in advance. | 2 |
| **Confusion matrix** | The table that counts the four outcomes of a classifier: true positives, false positives, false negatives and true negatives. | 0, 3 |
| **Context window** | The number of tokens a model can keep in view at one time, including your prompt, files and its answer so far. | 5, 9 |
| **Cosine similarity** | A measure of how closely two vectors point the same way: the dot product divided by the product of their lengths. 1 means the same direction. | 4 |
| **Cross-validation (k-fold)** | Split the data into k parts, train on k - 1, test on the held-out part, rotate so each part is the test part once, and average the scores. | 2 |
| **Data inventory** | A list of what data the organisation holds, where it is, who owns it and in what format. The first step of data readiness. | 14 |
| **Data quality dimensions** | The standard ways to judge data: completeness, accuracy, timeliness, consistency and representativeness. | 0, 14 |

| Term | In plain words | Session |
|---|---|---|
| **Decision tree** | A model made of yes-or-no questions on features. Each question is a split, each final answer a leaf, and depth is the number of questions on the longest path. | 2 |
| **Deep learning** | Machine learning with neural networks of many layers. It suits images, sound and language. | 0, 1 |
| **Drift** | A model getting worse over time because the world has changed since it was trained. | 0, 10 |
| **Embedding** | A vector, produced by a trained model, that represents the meaning of a text or an image, so that similar meanings sit close together. | 4, 9 |
| **Epoch** | One full pass over the training data. | 0, 1 |
| **Euclidean distance** | The straight-line gap between two vectors: the square root of the sum of the squared differences. For (3, 4) and (6, 8): 5. | 2, 4 |
| **F1 score** | The harmonic mean of precision and recall, so it stays near the weaker of the two: 2 x P x R / (P + R). | 3 |
| **Feature** | One input measurement the model uses, such as the floor area of a house. | 1, 2 |
| **Few-shot prompting** | A prompt that includes a few worked examples (shots) before the real request. Zero-shot gives only the instruction. | 8 |
| **Fine-tuning** | More training of an existing model on your own examples, so that it fits a particular style or task. | 9 |
| **Foundation model** | A large model pretrained once on broad data that many products are built on. A large language model (LLM) is one kind. | 6 |
| **Governance** | The rules, roles and checks that decide who may use AI, for what, and who answers when something goes wrong. | 13 |
| **Gradient descent** | Repeated small steps against the gradient of the loss, which lower the loss. The learning rate is the size of each step. | 0, 1 |
| **Guardrails** | Checks and limits placed around an AI system, such as input filters, output checks, spending limits and an off switch. | 11 |
| **Hallucination** | A confident answer from a model that is false or invented. | 10 |
| **Human-in-the-loop** | An approval step: a person approves, edits or rejects an action before it runs. | 11, 12 |
| **Inference** | Using a trained model to produce an answer for a new case. Training is learning; inference is using. | 0, 7 |
| **k-nearest neighbours (k-NN)** | Predict from the k most similar training examples, found with a distance metric: average their numbers, or take a majority vote. | 2 |
| **Label** | The correct answer attached to an example, used in supervised learning. | 2 |
| **Least privilege** | Give an agent the smallest set of permissions (scopes) that still lets it do the job. | 11 |
| **LoRA** | Low-rank adaptation: a fine-tuning method that trains a small adapter and leaves the original parameters unchanged. | 9 |
| **Loss (loss function, error)** | A number that measures how wrong the model's predictions are. Training tries to make it small. | 0, 1 |
| **Machine learning** | Computer systems that learn patterns from data instead of following only hand-written rules. | 0 |
| **Manhattan distance** | The gap measured along each axis and added: \|3 - 6\| + \|4 - 8\| = 7. Also called city-block distance. | 2, 4 |

| Term | In plain words | Session |
|---|---|---|
| **MCP (Model Context Protocol)** | An open standard for connecting an AI model to tools and data sources in a common way. | 11 |
| **Model** | The result of training: a function with adjustable parameters whose behaviour comes from the numbers it has learned. | 0 |
| **Neural network** | A model made of layers of neurons, loosely inspired by the brain. | 1 |
| **OCR** | Optical character recognition: reading the text from a scan or a photo. | 0, 14 |
| **Open weights** | A model whose trained parameters can be downloaded and run on your own machines. The licence may still limit use, and open source in the OSI sense needs more. | 7 |
| **Overfitting** | The model fits its training data, including its noise, and does worse on new data. Plainly: it has memorised its examples. | 0, 3 |
| **Parameters** | All the weights and biases of a model: the numbers that training adjusts. | 0, 1, 7 |
| **Permissions (scopes)** | What an agent or tool is allowed to read, write, send or spend. Listed in a permissions table. | 11 |
| **Pilot** | A small, time-limited trial in real work, with a measure of success set in advance. | 12 |
| **Post-training** | Training after pretraining to make a model useful: supervised fine-tuning (instruction tuning) and RLHF. | 6 |
| **Precision** | Of the cases the model flagged, the share that were truly positive: TP / (TP + FP). | 3 |
| **Pretraining** | The first stage of building a language model: learning to predict the next token from huge amounts of text. The result is a base model. | 5, 6 |
| **Prompt** | The input text you give a model. A system prompt holds standing instructions; a user prompt is the current request. | 8 |
| **Prompt injection** | Text hidden in an email, page or file, written to make an AI follow orders its owner never gave. | 10, 11 |
| **Quantisation** | Lowering the number of bits stored per parameter, for example to INT8 or 4-bit, so the model file is smaller. | 7 |
| **RAG (retrieval-augmented generation)** | A retriever fetches relevant passages from your documents and the model answers from them. | 9 |
| **Recall** | Of the truly positive cases, the share the model caught: TP / (TP + FN). | 3 |
| **Regression** | Predicting a number, such as a house price. | 2 |
| **Reinforcement learning** | Learning by trial and error, guided by rewards and penalties. | 0 |
| **RLHF** | Reinforcement learning from human feedback: people rate answers and the model is adjusted toward the ones they prefer. | 6 |
| **ROC curve** | Receiver operating characteristic: a curve of true positive rate against false positive rate at every threshold. | 3 |
| **ROI (return on investment)** | What you gain compared with what you spend, for example time saved against cost. | 12 |
| **Self-attention** | Each token weighs the earlier tokens by relevance, using a query, a key and a value. | 5 |
| **Softmax** | A function that turns the model's scores (logits) into probabilities that add up to 1. | 5 |
| **Supervised learning** | Learning from examples that come with the correct answer (the label). | 0, 2 |

| Term | In plain words | Session |
|---|---|---|
| **Temperature** | A setting that divides the logits before softmax: low gives safer, repeatable text and high gives more varied text. | 5, 8 |
| **Threshold** | The score at or above which a model says positive. Moving it trades false alarms against misses. | 3 |
| **Token** | A piece of text, a word or part of a word, that a language model reads and writes. A tokenizer cuts text into tokens from a fixed vocabulary. | 5 |
| **Training, validation and test sets** | Training: the model learns from it. Validation: used to compare models and tune settings. Test: used once at the end for an honest score. | 0, 3 |
| **Transformer** | The architecture of modern language models: a stack of identical blocks, each made of self-attention and a feed-forward network. | 5 |
| **Trigger** | The event that starts an automation, such as a new email arriving or a timer going off. | 12 |
| **Unsupervised learning** | Learning from data with no labels, by finding structure such as groups. | 0, 2 |
| **Vector, vector database** | A vector is a list of numbers. A vector database stores embeddings and finds quickly the ones closest in meaning to a question. | 4, 9 |
| **Weight** | A number that says how much one input counts. Training adjusts the weights. | 0, 1 |
| **Weighted sum** | Multiply each input by its weight and add the results: w1 x1 + w2 x2 + b, with the bias b added once. | 0, 1 |
| **Workflow** | A fixed series of steps, often across several apps, that runs when a trigger fires. | 12 |

Part 18

# The fourteen sessions at a glance

| No. | Session | You keep |
|---|---|---|
| 1 | Seeing: pixels, features and neurons | A one-page sheet: how a picture becomes numbers, a house price as a weighted sum, the umbrella neuron worked by hand, and the digit results |
| 2 | Learning from examples: types of machine learning, decision trees, nearest neighbours | A one-page sheet: the four types of learning, a decision tree read as rules, k-nearest neighbours, cross-validation and the honest scores |
| 3 | Measuring a model: accuracy, precision, recall, F1, ROC and AUC, thresholds, baselines, overfitting | The vendor question sheet: questions about the numbers behind any AI claim |
| 4 | Words as numbers: bag of words, TF-IDF, embeddings and distance | The distance cheat sheet: Euclidean, Manhattan, dot product and cosine similarity |
| 5 | Language models: tokens, attention and next-word probability | A one-page sheet: the next-token loop, the softmax recipe, temperature and five questions to ask about any answer |
| 6 | Foundation models: pretraining, post-training, model families, reasoning and multimodal models, benchmarks | A model comparison grid for your own tasks, with questions for a model card and a benchmark claim |
| 7 | Open weights, file formats, size and running cost | A rent, host or own decision sheet, with the memory sum and the licence questions |
| 8 | Prompting and reasoning models | A reusable prompt template and a prompt log |
| 9 | Giving a model your knowledge: retrieval-augmented generation, long context and fine-tuning | A decision tree: prompt first, then retrieve (RAG), then fine-tune |
| 10 | When models fail, and how to check them | A five-step check routine and a test-set template |
| 11 | Agentic AI: tools, loops, MCP, autonomy and guardrails | Your Tool Boundary Sheet, a permissions table: what an agent may read, write, send and spend, who approves, the log and the off switch |
| 12 | Workflow mapping (practical) | Your mapped workflow, the choice for each step, your assistant sketch and the hours-saved estimate in the Business Case Workbook |
| 13 | Governance and buying (practical) | Your Risk Register, AI Use Policy draft, Vendor Question List and questions for your lawyer |
| 14 | Data readiness assessment, roadmap and 90-day plan (practical) | Your inventory and scorecard, your roadmap, and the Follow-up Kit |

Session 0, the introduction, comes first and gives you this handbook. Each session lasts two hours. After the course, the instructor follows up at 30, 60 and 90 days.

Jotham Martin Wambi | KLAi Professional Learning | jothammartin.wambi@gmail.com | +256 750 990 718